

Laboratorio: comandos básicos de Docker (docker run e afíns)

Este laboratorio serve para practicar os comandos fundamentais de Docker. As imaxes e versións están especificadas en cada paso para garantir reproducibilidade.

1. Verificar o entorno Docker

Obxectivo: confirmar que Docker está correctamente instalado e operativo.

```
docker version
docker info
```

2. Descargar unha imaxe de Alpine Linux

Obxectivo: descargar manualmente a imaxe base alpine:3.20 dende Docker Hub sen executala.

```
docker pull alpine:3.20
```

3. Executar un contedor efémero

Obxectivo: lanzar un contedor de Alpine que mostre unha mensaxe e se elimine ao saír. Imaxe: alpine:3.20

```
docker run --rm alpine:3.20 echo "Ola dende Alpine en Docker!"
```

4. Abrir unha sesión interactiva

Obxectivo: entrar nun contedor Alpine para executar comandos Linux manualmente. Imaxe: alpine:3.20

```
docker run --rm -it alpine:3.20 sh
```

5. Executar un servidor Nginx básico

Obxectivo: levantar un servidor web usando a imaxe nginx:1.27-alpine. O contedor chámase web1 e expón o porto 8080 do host cara ao 80 do contedor.

```
docker run -d --name web1 -p 8080:80 nginx:1.27-alpine
```

6. Listar contedores activos

Obxectivo: comprobar que o contedor web1 está en execución.

```
docker ps
```

7. Ver os rexistros dun contedor

Obxectivo: consultar a saída de Nginx no contedor web1.

```
docker logs web1
```

8. Inspeccionar un contedor

Obxectivo: obter información detallada do contedor web1 (configuración, IP, montaxes...).

```
docker inspect web1  
docker inspect -f '{{.NetworkSettings.IPAddress}}' web1
```

9. Entrar no contedor en execución

Obxectivo: acceder á shell de web1 para explorar o sistema de ficheiros interno.

```
docker exec -it web1 sh
```

10. Copiar ficheiros ao contedor

Obxectivo: engadir unha páxina personalizada dentro de web1.

```
echo "<h1>Benvido ao meu sitio!</h1>" > index.html  
docker cp index.html web1:/usr/share/nginx/html/index.html
```

11. Executar outro Nginx en porto diferente

Obxectivo: levantar outro servidor web (web2) no porto 8081.

```
docker run -d --name web2 -p 8081:80 nginx:1.27-alpine
```

12. Usar variables de contorno

Obxectivo: lanzar un contedor de Alpine imprimindo o contido dunha variable. Variable: GREET=Ola

```
docker run --rm -e GREET="Ola" alpine:3.20 sh -c 'echo "$GREET, mundo!'"
```

13. Servir ficheiros locais (bind mount)

Obxectivo: servir un ficheiro HTML do host nun contedor Nginx. Imaxe: nginx:1.27-alpine, contedor: web3, porto: 8082. Directorio dende o que nginx serve os **htmls**: /usr/share/nginx/html.

```
mkdir -p site && echo "<h1>Contido local</h1>" > site/index.html
docker run -d --name web3 -p 8082:80 -v "./site:/usr/share/nginx/html:ro"
nginx:1.27-alpine
```

14. Volume nomeado persistente

Obxectivo: crear un volume permanente chamado webdata e usalo en web4. Imaxe: nginx:1.27-alpine, porto: 8083. Entrar no contedor e crear un index.html. eliminar o contedor e volver crealo para comprobar a persistencia.

```
docker volume create webdata
docker run -d --name web4 -p 8083:80 -v webdata:/usr/share/nginx/html nginx:1.27-
alpine
docker exec web4 sh -c 'echo "<h1>Páxina persistente</h1>" >
/usr/share/nginx/html/index.html'
docker stop web4 && docker rm web4
docker run -d --name web4b -p 8083:80 -v webdata:/usr/share/nginx/html nginx:1.27-
alpine
```

15. Limitar recursos de CPU e memoria

Obxectivo: executar un contedor con recursos limitados. Imaxe: alpine:3.20, nome: limited. limita a CPU a 0.5, a memoria 256 MB (256m) e executa o comando sleep 300.

```
docker run -d --name limited --cpus="0.5" --memory="256m" alpine:3.20 sh -c "sleep
300"
```

16. Consultar volumes e redes dispoñibles

Obxectivo: listar todos os volumes e redes creados.

```
docker volume ls
docker network ls
```

17. Comunicación entre contedores nunha rede

Obxectivo: crear unha rede chamada appnet e conectar dous contedores. redis-server → servizo Redis (imaxe redis:7-alpine) redis-client → contedor Alpine que executa redis-cli ping: apk add --no-cache redis && redis-cli -h redis-server ping

```
docker network create appnet
docker run -d --name redis-server --network appnet redis:7-alpine
docker run --rm --network appnet alpine:3.20 sh -c "apk add --no-cache redis &&
redis-cli -h redis-server ping"
```

Alternativa máis simple: empregar unha imaxe alpine e o comando sleep e conectarse a ela mediante ping.

```
docker run -d --rm --name host1 --network appnet alpine:3.20 sh -c "sleep 300"
docker run --rm --name host2 --network appnet alpine:3.20 sh -c "ping host1"
```

18. Eliminar contedores e volumes

Obxectivo: limpar recursos de práctica.

```
docker rm -f web1 web2 web3 web4b limited redis-server
docker volume rm webdata
docker network rm appnet
```

19. Limpeza final

Obxectivo: eliminar recursos non usados e liberar espazo.

```
docker system prune -f
```